

ons described
here.

GitLab LevelUp

GitLab LevelUp is a Learning Experience Platform (LXP) meant to provide training and professional development for team members. We will continue being handbook first, and use this platform to organize, surface, and promote relevant content that exists today.

SECTION: engineering/training/ic-to-manager.md

Transitioning from IC to a Manager Training Video

```
<figure class="videocontainer">  
  <iframe src="https://www.youtube.com/embed/Zeull-tdy6o" frameborder="0"  
allowfullscreen="true"> </iframe>  
</figure>
```

Is Management right for me?

The best way to determine whether or not management is the track you would like to pursue, is to learn more about what managers do through online training and discussions with Managers.

Online Training

There are some online training courses that describe the managerial experience:

- Elevate Manager Training
- New Manager Foundations
- Succeeding as a First Time Manager
- Management Foundations

Role Playing Manager Real Life Manager Situations

Role Playing is a very useful tool for a Manager to share Managerial situations with Individual Contributors who are interested in becoming a Manager.

- Q&A with your manager after role playing
- Take on projects that are less procedural to practice those skills

Learn More about the journey from Individual Contributor to Manager

Review Promotion Docs for team members from your functional area who have recently been promoted to Engineering Managers

Schedule a coffee chat with team members who have recently been promoted and ask them questions that will help you learn more about their transition and their new role

- See what is expected of GitLab leadership

- Consider a mentor outside of GitLab

Stay Informed

- Join the Engineering Managers Slack Channel
- Read the Engineering Week In Review weekly

Exceed in Your Current Role

- Make sure that you exceed in your current role so you have the technical skillset proven for the next level

- Exceed consistently for a long period of time. See the performance/potential matrix.

- Have patience - growing into a management role takes time

- Ask your manager what the gap is between where you are and where you want to be, and get a plan in place to grow. If your manager does not have a plan, ask for one and provide valid feedback about the timeline

- Get a mentor to help you grow to where you want to be

- Take the hard tasks that no one wants

- How can I gain Managerial experience?

- Currently as an Individual Contributor, your day to day tasks probably do not include management type activities. Below is a list of activities you can engage in to get managerial experience without actually being a manager at GitLab.

Back Up Your Manager

- Serve as a Backup Engineering Manager when your Engineering Manager is Out of the Office, which could include release planning, weekly meetings, and incident coordination

- Attend public meetings with your Manager taking notes (if necessary) and filling in when your manager is Out of the Office

- Monitor the eng-manager Slack channel for opportunities to work on Managerial related tasks

- Support the Engineering Manager with OKR related activities (Creating, Scoring , Updating)

- Participate in the weekly triage issue analysing incoming bugs for your team

- Monitor the is-this-known Slack channel and create issues of unidentified bug reports

Hiring

- Participate in Interviews

- Review issue roadmaps and priorities to identify capacity problems

- Serve as an Onboarding Buddy

Growing Others

- Mentor a team member and host regularly scheduled 1-1's with a Mentee

- Regularly provide feedback in mentoring relationships, team retrospectives, issues, thanks channel and Team Slack Channels.

- Host a Book Club

Initiate impactful change

- Lead a team or GitLab initiative to improve quality, security, or performance

Identify an organizational improvement and drive it to completion, such as facilitating training or the first responder program.

Volunteer whenever the organization is asking for Volunteers, be the first to raise your hand

Strengthen your Efficiency Value

Work more efficiently by analyzing what you do on a daily basis and improve your own processes

Identify and implement process improvements that improve your team's efficiency

Identify and implement process improvements that improve your efficiency that impacts others outside of your team

Strengthen your Diversity Value

Join Diversity Inclusion & Belonging or TMRG Slack Channel(s)

Join a TMRG

Attend TMRG Hosted Events

Participate in TMRG or DIB related events / initiatives

Participate in GitLab Meetings

Have regular meetings with counterparts in security, tech writing, quality, so on.

Volunteer to host a GitLab Team retrospective

Volunteer to be the on-call coordinator

Aspiring Manager mentorship pilot program

Background

At GitLab, there are a number of ICs that are interested in learning and growing into a Management role.

A number of resources have been assembled to aid in the process of understanding the manager role and to gain the experience required to take it on.

While resources are valuable in their own right, having an experienced Manager as a mentor can help guide ICs and provide practical advice and background. These Manager-mentors can aid the IC and shed light on what areas of Management they could focus on and what they can do practically at present to skill-up.

Structure

1. Participation is voluntary for both Managers and Aspiring Managers.

1. Managers may directly mentor up to 4 Aspiring Managers at a time.

1. Manager / Aspiring Manager mentorships are coordinated within the `managermentorship.yml` file.

1. Aspiring Managers should locate a Manager that has an opening available in `managermentorship.yml` and contact them directly.

1. Manager and Aspiring Managers should check in at least every 6 weeks. This could happen async or via Coffee chat.

1. Discussion topics would be completely flexible based on needs and interests.

Benefits

1. Develops and expands mentorship skills among Managers
1. Gives Aspiring Managers a regular touch point for skilling-up in deficient areas
1. Creates stronger networks amongst Managers / Aspiring Managers than currently exists
1. Mentoring directly under Managers should catalyze more competence and confidence in understanding what a Management role at GitLab looks like and how to succeed.

Measuring Success

In order to measure the effectiveness of the mentorship program, a quarterly anonymous survey will be conducted for both Manager and Aspiring Managers.

Additionally, a retrospective conversation will occur 6 months after the initiation of the program aiming to gather feedback on the pilot and determine whether it should continue.

SECTION: engineering/training/reliability.md

Goal of this training

As we have a renewed focus on reliability in engineering to reduce outages, we have made many changes to the handbook, production documentation, and our processes. While we have announced them via multimodal communication (EWIR, slack, email, meetings), not everyone has likely seen and internalized all of the important changes.

We want to gather all the crucial changes, explain why we made them, discuss a summary, and link to where you can find more information.

This material is available as a learning pathway on GitLab's Level Up.

Introduction

```
<!-- blank line -->
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/Evhtsds-l-Q" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->
```

Amplifying SaaS Reliability Focus

Reliability & Security Standup

The business impact of reliability

```
<!-- blank line -->
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/XJ7Q771IzvY" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
```

<!-- blank line -->

Importance of reliability to the business

Impact of reliability on users

Video (not public)

Improving SUS - slides 9 through 14 in particular

Updates to values

<!-- blank line -->

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/IBpGgl3IEz4" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->
```

MR to change quality and reliability

MR around things that don't scale

Blameless Culture

<!-- blank line -->

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/WMkchw5hzlc" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->
```

Google SRE Book: Blameless culture

Limiting the impact of far reaching work

<!-- blank line -->

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/bHukJTipMJk" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->
```

Limiting the impact of far-reaching work

Overview of Risk Mapping

<!-- blank line -->

```
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/-i0bFAznPGI" frameborder="0"
```

allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->

Developer Experience Risk Mapping

MR acceptance checklist

<!-- blank line -->
<figure class="videocontainer">
 <iframe src="https://www.youtube.com/embed/yrOrueDn1es" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->

MR acceptance checklist

Updates to the definition of done

<!-- blank line -->
<figure class="videocontainer">
 <iframe src="https://www.youtube.com/embed/cQVPjW-wakQ" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->

Definition of Done

Backwards Compatibility

<!-- blank line -->
<figure class="videocontainer">
 <iframe src="https://www.youtube.com/embed/6OQWyc1M8Q" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->

Course on backwards compatibility

How to use the stage group dashboards to understand how a feature category performs

<!-- blank line -->
<figure class="videocontainer">
 <iframe src="https://www.youtube.com/embed/WGctFcKjGsY" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->

Stage group dashboard documentation

Error budgets

<!-- blank line -->

<figure class="videocontainer">

<iframe src="https://www.youtube.com/embed/wmOSPGB03rs" frameborder="0" allowfullscreen="true"> </iframe>

</figure>

<!-- blank line -->

Error budgets

Feature Change Locks (FCL)

<!-- blank line -->

<figure class="videocontainer">

<iframe src="https://www.youtube.com/embed/jO-hucp2yg" frameborder="0" allowfullscreen="true"> </iframe>

</figure>

<!-- blank line -->

Feature change locks

Added past due infradev as a KPI

<!-- blank line -->

<figure class="videocontainer">

<iframe src="https://www.youtube.com/embed/cZVOCIfsI8" frameborder="0" allowfullscreen="true"> </iframe>

</figure>

<!-- blank line -->

Past due infradev issues

Overview of Engineering Metrics Dashboards

<!-- blank line -->

<figure class="videocontainer">

<iframe src="https://www.youtube.com/embed/vOsyw473M1M" frameborder="0" allowfullscreen="true"> </iframe>

</figure>

<!-- blank line -->

Engineering metric dashboards

Feedback on the training

What did you like about the training?

What did you not like that we should improve?

Add your comments in this feedback issue.

SECTION: engineering/training/staff-engineers.md

Staff Engineers

You became a Staff Engineer! Now what?

This is an amazing accomplishment! Staff promotions are difficult to achieve because you have to be acting in any "next" role at some capacity to support a promotion to that role, and the Staff has a larger scope to be showing those behaviors than other roles. For this reason, you may feel like you've already been doing the role for a long time - but there are more opportunities available and potentially more to learn!

Changes to expect in your new role

One difference between Staff and Senior positions is the leadership aspect that Staff engineers must exhibit and grow into. Leadership is not for everyone, so it makes sense that many senior engineers would prefer to stay in their positions long-term. As you transition more fully into your new Staff role, you will:

- Shift your technical prowess from hands-on to hands-off by guiding, coaching, aligning, and organizing others. You will work on the most needed, critical, or complex topics within your group and be depended on to save the group when they are struggling, but otherwise you will work to empower others to take your place on the topics you were covering as a strong senior.
- Be doing a lot of context switching - it's difficult to help people and "get your job done" at the same time. Remember, it is now your job as a leader to help people too! Context switching and time management are skills that leaders can struggle with.
- Be an example to others - in your responses (disagreements or agreements), proposals, and in the way you work or further yourself, keep in mind that others may be referring to you when they think about how they should handle a situation.
- Practice peer accountability by holding others accountable to their commitments and guiding them toward improvement areas

Another difference is the gap between Staff and Principal - a Staff engineer is the foremost engineer within their group, and a Principal engineer is the equivalent within their stage. This means that more complex responsibilities may be coming to you across the stage or from other sub-departments, and you may need to adapt to this change. Remember, a Staff engineer is not completely different from a Senior or Principal - you may accomplish Senior requests quicker and Principal requests slower, but still be asked to do them.

Tactical initiatives for a Staff engineer

- Becoming a maintainer of more projects, specialties, or technologies
- Become an owner of the triage report, assessing the severity of incoming issues and advocating for the priority they should take
- Begin formally mentoring 1-2 others in their own careers
- Being a backup for your manager, and working directly with counterparts on direction and vision
- Facilitate a working group or join as a functional leader

- Participate in the CEO Shadow Program
- Advocate for improvements to processes or initiatives that impact the wider organization as a whole (Sub-Department, Department, Company - Impacts to KPI improvements, Values, Quality, Security, Product, Performance, Definition of Done, Etc)
- Participate more frequently in Diversity, Inclusion, and Belonging initiatives
- Become a DRI for architectural initiatives or architectural blueprints where relevant
- Break down large tasks for the team into epics with smaller issues or MVCs
- Represent your group to leadership by speaking to engineering allocation status, reliability standup topics, iteration office hours, etc
- Conducting behavioral and technical interviews, or other hiring topics (filtering candidates, recommending ways to decrease bias or increase diversity)
- Becoming a reviewer mentor
- Conducting RCAs or Iteration Retrospectives
- Collaborate directly with customers by participating in technical discussions to understand use-cases or troubleshooting
- Collaborating with security on threat modeling for your area, infrastructure on production readiness initiatives, product on long-term implementation strategies, other teams on integrations and delivery
- Documenting best practices for your group, stage, sub-department, or department
- Working across teams on global initiatives (collaborating on a decision to migrate to Pajamas and enacting a plan, setting global GraphQL, API, pagination standards)

SECTION: engineering/training/staff-plus-role.md

Overview

This page provides resources for Staff+ engineers and aspiring Staff engineers.

How to stay up to date

As Staff+ engineer it's important to stay up to date with what's happening in engineering. By staying up to date you can better understand where problems and inefficiencies can be and what we could do or explore to make things better.

Here are various activities that can be useful for this purpose:

See the Engineering Communications page.

Read the release posts every month. It helps understand what other team have worked on. What features we have added.

Keep an eye on slack channels (architecture, backend, development, staffplusengineers, etc.).

It is important have a basic understanding of what problems a particular technology, pattern, process is meant to solve.

This way we can better understand if it can be applicable to other scenarios.

Take opportunity during maintainer reviews to understand new features, patterns, tools and to scan for interesting aspects.

There is a surprising amount of knowledge we can gain doing just that, which often allows us to discover opportunity for reuse between MRs.

Keep an eye on new and existing architectural design docs. Pay closer attention to docs around

your areas of expertise while acknowledging the existence of other ones.

Use some of your time for deliberate learning.

Keep a list of things you want to learn more and where you feel there can be potential impact.

It could be a technology (e.g. Clickhouse), a tool (e.g. Packwerk), a feature (e.g. Audit Events).

Book club

We hosted a book club: The Staff Engineer's Path (internal link), where you can find more resources.

SECTION: engineering/training/em-role/_index.md

Engineering manager role explained

Introduction

The goal of this page is to provide bite-sized materials for Engineers who are interested in the Engineering Manager role.

An IC's guide to managerial work

A short article about What does your manager do all day? What would it mean to become one?.

Videos

In the videos below different team members explained intricacies of the EM role, specifically from the perspective of the Engineer and transitioning from the Individual Contributor track to the Management track.

All videos are also available in the audio format here.

Michelle

<!-- blank line -->

<figure class="videocontainer">

<iframe src="https://www.youtube.com/embed/4TzP1R3dfSk" frameborder="0" allowfullscreen="true"> </iframe>

</figure>

<!-- blank line -->

Jarka

<!-- blank line -->

<figure class="videocontainer">

```
<iframe src="https://www.youtube.com/embed/U91uIREoqQE" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->
```

Brandon

```
<!-- blank line -->
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/m3w9JMqOQVM" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->
```

Nicolas - Transitioning from EM to IC

```
<!-- blank line -->
<figure class="videocontainer">
  <iframe src="https://www.youtube.com/embed/Um4-C2RC0uo" frameborder="0"
allowfullscreen="true"> </iframe>
</figure>
<!-- blank line -->
```

SECTION: engineering/training/em-role/work_of_a_manager.md

TOC

The what and the why

What does your manager do all day? What would it mean to become one?

This document is about the what (and a bit about the why). It would take an entirely different - and much longer - conversation to talk about how to do each of these things well.

The summary: a manager's job is to build an organization that performs well, efficiently, and sustainably. (I've heard and used the term well oiled machine.) This is especially important to internalize for ICs (individual contributors, meaning not managers) considering switching to the manager role: management is not on top of your daily duties; it's an entirely different job.

Can you still do a bit of that old day job, like coding? Sure, but that's a "nice to have" from the perspective of the company. It may be essential to your sanity, so by all means dabble, but this dabbling must not take precedence. Your job is to enable people and build healthy, focused organizations..)

This document is a framework

Use this document to think about a manager's work. It's intended to focus thought and break up

the scope into manageable pieces. It is not intended to be prescriptive or exhaustive as to what you should do. The questions and exercises in this document may or may not have an immediate answer. They're meant to raise questions that you may wish to revisit from time to time.

Read through and discuss the contents, as well as the results of the exercises, with a mentor or peer manager.

The matrix

Managers build the "well-oiled machine" by taking care of their direct and indirect reports, and their larger organization. They must align what the business needs, and what the team (and individuals) need. They must also care for themselves, and ensure growth and sustainability. It's essential for managers to balance day-to-day work with forward-looking work. The needs of the now must be balanced with the needs of upcoming large projects, long-term career planning, staffing, and more.

Use this matrix to organize and visualize these concepts:

	Self	Individuals	Organization	Work
Short term				
Medium term				
Long term				

The columns

Self: Processes relevant to yourself. Improving yourself as a manager, avoiding burnout, and working on your own career. Also meta-work that enables you to function, from mandatory trainings to tracking your achievements.

Individuals: Processes relevant to individuals. The lives of the people that report to you (directly or indirectly), and any mentees in your charge. Their happiness, skills, careers, goals, and individual productivity.

Organization: Processes relevant to broader collaboration: your team, or your whole organization. Interactions of people, organizational culture and structure, and how the team fits into the overall company landscape.

Work: Processes relevant to the business and its overall goals. OKRs, roadmaps, and the team's deliverables. Picking, planning and organizing what work you will be doing - and, just as importantly, not doing.

The rows

Short term: Daily work. Something you need to do to keep rolling. All input is already at hand. Effects are immediate.

Medium term: Work that requires you to look forward or backward a few weeks or months. Use that broader perspective to do something, or lay a foundation for future work. Effects are usually realized later.

Long term: Work that requires you to look beyond current commitments (often 6-12 months) to understand long-term needs. Drive strategy, prepare for eventualities, or lay a foundation to prepare you for this work. Effects are usually realized much later, although sometimes long-

term goals can become urgent unexpectedly.

Exercise 1: activities in detail

Take five (okay, 15) minutes and put some activities into each box. Think about why: if it's an activity done to keep individuals happy and productive (like actually talking to them from time to time) add it to box Short term, Individuals.

No, really. · You'll see mine after.

<details markdown="1">

<summary>My example Matrix (click to expand)</summary>

This is of course not an exhaustive list, but an illustration.

Short term, Self

- Keep notes of my daily activities
- Process my input stream (Slack, email, issue updates) for planning
- Take time off, update calendar, PTO issues etc

Short term, Individuals

- 1:1s: staying informed, ensure I'm available for helping
- Read Slackbot updates
- Process information into plans, and spot trends
- Administrivia such as expense reports, getting new team members their laptops
- Actual short-term help: "glue" work to connect people, projects and ideas
- Actual logistics of talent assessment and promotions

Short term, Organization

- Conduct team meetings
- Process information into plans, and spot trends
- Help resolve any misunderstandings
- Actual hiring process

Short term, Work

Schedule work that's already been decided (like release planning) or make decisions based on existing roadmap and vision

- Follow up on key issues, if need be, and unblock
- Ensure that tasks are properly staffed by the correct people

Medium term, Self

- Maintain my prioritized TODO list
- Grow and maintain my network. Notice opportunities and foster relationships
- Collect medium-term feedback (such as upon project completion, or interaction finished)

Medium term, Individuals

- Mentor and coach
- Spot recognition for good work (like thanks channel and discretionary bonuses)
- Keep an eye on continued performance

Medium term, Organization

- Plan team events
- Plan trainings needed
- Review productivity over time, and foster improvements where needed
- Process input from recurring meetings on different levels, and generate tasks

Medium term, Work

- Plan team events
- Mid-term planning: what should we do next quarter that we aren't doing now?
- Project staffing: who's going to work on that?
- OKR progress tracking, and interventions if need be
- Long term, Self
 - Plan team events
 - Strategy plans, self. (Where do I want to be in five years?)
 - Career planning, self. (How do I get there?) Gap analysis
 - Education, self. Trainings, books, mentoring
 - Collect long-term feedback
- Long term, Individuals
 - Plan team events
 - Career planning for individuals, balancing their needs and wants versus available opportunities
 - Set people up for success: champion their goals, get them "in the room" for relevant discussions, get them recognition for long-term results
 - Happiness planning long term: how will work fit into their overall lives?
- Long term, Organization
 - Plan team events
 - Reassess organizational structure and fit into the larger org: what serves this team best?
 - Reassess skill mix: what do we need that we don't have?
 - Plan for staffing
 - Team and organization culture planning: what should change, what should be encouraged
 - Reassess team and organization values
 - Get recognition of and visibility for team results
 - Long-term feedback loops, and action items resulting from them
- Long term, Work
 - Strategic plans: keeping vision and roadmaps up-to-date
 - Translate company business goals into organization-level, actionable project ideas
 - Bring bottom-up initiatives to higher-level attention

</details>

Exercise 2: relative importance of activities

Say you have some tasks ready to go in each box.

Where should you focus your attention? What about over a longer period?
 What happens if you don't do some of this for a day, a month, or a year?
 How do you ensure that nothing important is forgotten?

Exercise 3: quality control

Where in the matrix was your attention in the recent past? Where wasn't it?

What was the reason?
 What was the effect?
 What are some situations that require more focus on one box, rather than another?

References

Transitioning to a manager role
Becoming a GitLab manager issue
Elevate training program
High Output Management
How to be a great remote manager

SECTION: engineering/workflow/_index.md

This document explains the workflow for anyone working with issues in GitLab Inc.
For the workflow that applies to the wider community see the contributing guide.

GitLab Flow

Products at GitLab are built using the GitLab Flow.

We have specific rules around code review.

Reverting a merge request

In line with our values of short toes, making two-way-door decisions and bias for action, anyone can propose to revert a merge request. When deciding whether an MR should be reverted, the following should be true:

- Something broke and there is no acceptable work around. Examples of this include:
 - A feature broke and is categorized as ~severity::1 or ~severity::2.
See severity labels
 - Master broken
 - There are failing migrations
- There are no dependencies on the change. For example, a database migration has not been run on production.

Reverting merge requests that add non-functional changes and don't remove any existing capabilities should be avoided in order to prevent designing by committee.

The intent of a revert is never to place blame on the original author. Additionally, it is helpful to inform the original author so they can participate as a DRI on any necessary follow up actions.

The pipeline::expedited label, and master:broken or master:foss-broken label must be set on merge requests that fix master to skip some non-essential jobs in order to speed up the MR pipelines.

Broken master

If you notice that pipelines for the master branch of GitLab or GitLab FOSS are failing,

returning the build to a passing state takes priority over everything else development related, since everything we do while tests are broken may:

- break existing functionality
- introduce new bugs and security issues
- impede productivity for all of engineering and our release processes

What is a broken master?

A broken master is an event where a pipeline in master is failing.

The cost to fix test failures increases exponentially as time passes due to merged results pipelines used. Auto-deploys, as well as monthly releases and security releases, depend on gitlab-org/gitlab master being green for tagging and merging of backports.

Our aim should be to keep master free from failures, not to fix master only after it breaks.

Any question or suggestion is welcome in the gdevelopmentanalytics channel who owns the broken master automation proceess.

Broken master service level objectives

There are two phases for fixing a broken master incident which have a target SLO to clarify the urgency. The resolution phase is dependent on the completion of the triage phase.

| Phase | Service level objective | DRI |

| --- | --- | --- |

| Triage | 4 hours from the 2nd occurrence of a broken master incident creation until assignment | group labeled on the incident |

| Resolution | 4 hours from assignment to DRI until incident is resolved | Merge request author or team of merge request author or dev on-call engineer |

Note: Recurring incidents are negatively impacting master pipeline stability and development velocity. Any untriaged, recurring incident will be automatically escalated to dev-escalation following this timeline:

mermaid

timeline

title Pipeline incident escalation

section Pipeline failure incident 1

not recurring in 24 hours and no human activity : Auto closed

any human update on incident 1

: labels incident 1 with escalation skipped

: does not trigger any group ping or escalation

same job failures recurring in incident 2

: closes incident 2 as duplicate of incident 1

: labels incident 1 with escalation needed

: pings attributed group channel after 10 minutes of inactivity

: 2nd ping to group channel after 30 minutes of inactivity

: pings stage channel in after 3 hours 40 minutes of inactivity

- : escalates to dev-escalation after 4 hours of inactivity
- : labels incident 1 is escalated

If an incident becomes a blocker for MRs and deployments before being auto-escalated, the team member being impacted should refer to the broken master escalation steps to request help from the current engineer on-call as early as needed.

Additional details about the phases are listed below.

Broken master escalation

Recurring broken master incidents are automatically escalated to dev-escalation unless it is triaged within 4 hours.

If a broken master is blocking your team before auto-escalation (such as creating a security release) then you should:

1. See if there is a non-resolved broken master incident with a DRI assigned and check discussions there.
1. Check discussions on the failure notification in the triage DRI's group Slack channel to see if anyone is investigating the incident you are looking at. See Triage broken master for information on who the triage DRI is.
1. If there is not a clear DRI or action to resolve, use the dev escalation process to solicit help in the broken master incident.

Escalation on weekends and holidays

Master broken incidents must be manually escalated to dev-escalation on weekends and holidays if necessary. Without a manual escalation, the service level objective can extend to the next working day; that is, triage DRI is expected to triage the incident on the next working day. Regardless of when the label was applied, we always consider an incident to be in an escalated state as long as it has the `~"escalation::escalated"` label, until the incident is resolved.

Triage broken master

Definitions

- Flaky test: A test that fails, then succeeds when the CI job running the test is retried.
- Broken master:
 - A test that fails even when the CI job running the test is retried.
 - A failing test that can be reproduced locally on the master branch.

Attribution

If a failed test can be traced to a group through its `featurecategory` metadata, the broken master incident associated with that test will be automatically labeled with this group as the triage DRI through this line of code. In addition, Slack notifications will be posted to the group's Slack channel to notify them about ongoing incidents. The triage DRI is responsible for monitoring, identifying, and communicating the incident.

A notification will be sent to the attributed group's Slack channel and master-broken.

Triage DRI Responsibilities

1. Monitor

- Pipeline failures are sent to the triage DRI's group channel, if one is identified, and will be reviewed by its group members. The failures will also be sent to master-broken for extra communication. If an incident is announced in a DRI group's Slack channel, the channel member should acknowledge it and assume the triage DRI responsibilities.

- If the incident is a duplicate of an existing incident, use the following quick actions to close the duplicate incident:

```
shell
/assign me
/duplicate <originalissueid>
/copymetadata <originalissueid>
```

- If the incident is not a duplicate, and needs some investigation:
 - Assign the incident to yourself: /assign me
 - Change the incident status to Acknowledged (in the right-side menu).
 - In Slack, the :ack: emoji reaction should be applied by the triage DRI to signal the linked incident status has been changed to Acknowledged and the incident is actively being triaged.

1. Identification

- Review non-resolved broken master incidents for the same failure. If the broken master is related to a test failure, search the spec file in the issue search to see if there's a known failure::flaky-test issue.

- If this incident is due to non-flaky reasons, communicate in development, backend, and frontend using the Slack Workflow.

- Announce that master is fixed by enter /broadcast master fixed in the chat bar of the master-broken channel to invoke this workflow, and then click Continue the broadcast.

- Create a revert MR directly to save some time in case we need to revert down the line.

- If you are reverting an MR that performs a database migration, you need to follow the Deployment blockers process to prevent the migration from proceeding to deploy and running on staging and production.

- If the migration is executed in any environments, communicate to the release managers in releases channel and discuss whether it's appropriate to create another migration to roll back the first migration or turn the migration into a no-op by following Disabling a data migration steps.

- If you identified that master fails for a flaky reason, and it cannot be reliably reproduced (i.e. running the failing spec locally or retrying the failing job):

- Quarantine the failing test to restore pipeline stability within 30 minutes if the flakiness is continuously causing master pipeline incidents.

- Alternatively, if the failure does not seem disruptive, and you have a fix that you are confident with, submit the fix MR with the ~"master:broken" label to ensure your pipeline is expedited.

- If a flaky test issue already exists, add a comment in it with a link to the failed broken master incident and/or failed job. We have automation in place to create test failure

issues automatically. The issue is named after the spec path, which can be a search keyword.

- If a flaky test issue doesn't exist, create an issue from the New issue button in top-right of the failing job page (that will automatically add a link to the job in the issue), and apply the Broken Master - Flaky description template.

- Add the appropriate labels to the main incident:

```
shell
```

```
Add those labels
```

```
/label ~"master-broken::flaky-test"
```

```
/label ~"failure::flaky-test"
```

```
Pick one of those labels
```

```
/label ~"flaky-test::dataset-specific"
```

```
/label ~"flaky-test::datetime-sensitive"
```

```
/label ~"flaky-test::state leak"
```

```
/label ~"flaky-test::random input"
```

```
/label ~"flaky-test::transient bug"
```

```
/label ~"flaky-test::unreliable dom selector"
```

```
/label ~"flaky-test::unstable infrastructure"
```

```
/label ~"flaky-test::too-many-sql-queries"
```

- Close the incident

- Add the stacktrace of the error to the incident (if it is not already posted by gitlab-bot), as well as Capybara screenshots if available in the job artifacts.

- To find the screenshot: download the job artifact, and copy the screenshot in artifacts/tmp/capybara to the incident if one is available.

- Identify the merge request that introduced the failures. There are a few possible approaches to try:

- Check the commit in the failed job, and find the associated MR, if any (it's not as simple most of the times though).

- Look at the project activity, and search for keywords in the recent merged events.

- Look at the recent commits on master and search for keywords you might see in the failing job/specs (e.g. if you see a geo spec file is failing, specifically the shard spec, search for those keywords in the commit history).

- You can filter with the Merge branch text to only see merge commits.

- Look at the spec file history or blame views, by clicking respectively the History or Blame button at the top of a file in the file explorer, e.g. at <https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/backup.rb>.

- If you identified a merge request, assign the incident to its author if they are available at the moment. If they are not available, assign to the maintainer that approved/merged the MR. If none are available, mention the team Engineering Manager and seek assistance in the development Slack channel.

- You can find the team somebody is in and who's the manager for that team by searching in <https://handbook.gitlab.com/handbook/product/categories/>.

- If no merge request was identified, ask for assistance in the development Slack channel.

- Please set the appropriate ~master-broken: label from the list below:

```
shell
```

```
/label ~"master-broken::caching"
```

```
/label ~"master-broken::ci-config"  
/label ~"master-broken::dependency-upgrade"  
/label ~"master-broken::external-dependency-unavailable"  
/label ~"master-broken::flaky-test"  
/label ~"master-broken::fork-repo-test-gap"  
/label ~"master-broken::pipeline-skipped-before-merge"  
/label ~"master-broken::test-selection-gap"  
/label ~"master-broken::need-merge-train"  
/label ~"master-broken::gitaly"  
/label ~"master-broken::state leak"  
/label ~"master-broken::infrastructure"  
/label ~"master-broken::infrastructure::failed-to-pull-image"  
/label ~"master-broken::infrastructure::frunner-disk-full"  
/label ~"master-broken::infrastructure::gitlab-com-overloaded"  
/label ~"master-broken::job-timeout"  
/label ~"master-broken::multi-version-db-upgrade"  
/label ~"master-broken::missing-test-coverage"  
/label ~"master-broken::undetermined"
```

1. (Optional) Pre-resolution

- If the triage DRI believes that there's an easy resolution by either:
 - Reverting a particular merge request.
 - Making a quick fix (for example, one line or a few similar simple changes in a few lines).

The triage DRI can create a merge request, assign to any available maintainer, and ping the resolution DRI with a @username FYI message.

Additionally, a message can be posted in backendmaintainers or frontendmaintainers to get a maintainer take a look at the fix ASAP.

- If the failures occur only in test-on-gdk jobs, it's possible to stop those jobs from being added to new pipelines while the cause is being fixed. See the runbook for details.

Pro-tips for Triage DRI

1. For an initial assessment of what might have contributed to the failure, we can try the experimental AI-assisted root cause analysis feature following this documentation.
2. To confirm flakiness, you can use the @gitlab-bot retryjob <jobid> or the @gitlab-bot retrypipeline <pipelineid> command to retry the failed job(s), even if you are not a project maintainer.

- Note, The retryjob command can fail for the following reasons:
 - Retrying the same job twice with the retryjob command will result in a failure message because each failed job can only be retried once.
 - If there is no response to either of the retry commands, you are likely invoking them in non-supported projects. If you'd like to request for the commands to be added to your project, please make an issue and inform gdevelopmentanalytics. You are encouraged to self-serve the MR following this example and submit it for review for maximum efficiency.

Resolution of broken master

The merge request author of the change that broke master is the resolution DRI.

In the event the merge request author is not available, the team of the merge request author will assume the resolution DRI responsibilities.

If a DRI has not acknowledged or signaled working on a fix, any developer can take assume the resolution DRI responsibilities by assigning themselves to the incident.

Responsibilities of the resolution DRI

1. Prioritize resolving recurring broken master incidents over new bug/feature work. Resolution options include:

- Default: Revert the merge request which caused the broken master. If a revert is performed,
 - create an issue to reinstate the merge request and assign it to the author of the reverted merge request.
 - Reverts can go straight to maintainer review and require 1 maintainer approval.
 - The maintainer can request additional review/approvals if the revert is not trivial.
 - The pipeline::expedited label, and master:broken or master:foss-broken label must be set on merge requests that fix master to skip some non-essential jobs in order to speed up the MR pipelines.
- Quarantine the failing test if you can confirm that it is flaky (e.g. it wasn't touched recently and passed after retrying the failed job).
- Add the quarantined test label to the failure::flaky-test issue you previously created during the identification phase.
- Create a new merge request to fix the failure if revert is not possible or would introduce additional risk. This should be treated as a priority::1 severity::1 issue.
- To ensure efficient review of the fix, the merge request should only contain the minimum change needed to fix the failure. Additional refactor or improvement to the code should be done as a follow-up.

1. The resolution DRI must address all failures in the pipeline. Be mindful that the initial opened issue for the incident will only announce the jobs that failed so far. But after you fix those jobs, other subsequent jobs could fail on the same pipeline that you're triaging. The triage DRI is responsible for this whole pipeline, and not only for the initial failed jobs.

1. Apply the Pick into auto-deploy label (along with the needed severity::1 and priority::1) to make sure deployments are unblocked.

1. If the broken master incident affects any stable branches (e.g. <<https://gitlab.com/gitlab-org/gitlab/-/mergerequests/25274>>) or is caused by a flaky failure, open new merge requests directly against the active stable branches and ping the current release manager in the merge requests to avoid delays in releases / security releases.

See How to fix a broken stable branch guide for more details.

1. Communicate in master-broken when the fix was merged

1. Once the incident is resolved, select the Broadcast Master Fixed workflow in the master-broken channel, and click Continue the broadcast to communicate it.

1. When master build was failing and the underlying problem was quarantined / reverted / temporary workaround created but the root cause still needs to be discovered, the investigation should continue directly in the incident.

1. Create an issue for the Development Analytics group describing how the broken master incident could have been prevented in the Merge Request pipeline.

1. When resolution steps are completed and all of the required fixes are merged, close the incident.

Responsibilities of authors and maintainers

Once the resolution DRI announces that master is fixed:

- Maintainers should start a new merged results pipeline (for canonical MRs) and enable "Auto-merge".
There's no need to rebase once master has been fixed since we use merged results pipelines.
- (For forks only) Authors should rebase their open merge requests (since merged results pipelines isn't supported in these cases).

Merging during broken master

Merge requests can not be merged to master until the incident status is changed to Resolved.

This is because we need to try hard to avoid introducing new failures, since it's easy to lose confidence if it stays red for a long time.

In the rare case where a merge request is urgent and must be merged immediately, team members can follow the process below to have a merge request merged during a broken master.

Criteria for merging during broken master

Merging while master is broken can only be done for:

- Merge requests that need to be deployed to GitLab.com to alleviate an ongoing production incident.
- Merge requests that fix broken master issues (we can have multiple broken master issues ongoing).

How to request a merge during a broken master

First, ensure the latest pipeline has completed less than 2 hours ago (although it is likely to have failed due to gitlab-org/gitlab using merged results pipelines).

Next, make a request on Slack:

1. Post to either the frontendmaintainers or backendmaintainers Slack channels (whichever one is more relevant).
1. In your post outline why the merge request is urgent.
1. Make it clear that this would be a merge during a broken master, optionally add a link to this page in your request.

Instructions for the maintainer

A maintainer who sees a request to merge during a broken master must follow this process.

Note, if any part of the process below disqualifies a merge request from being merged during a broken master then the maintainer must inform the requestor as to why in the merge request (and optionally in the Slack thread of the request).

First, assess the request:

1. Add the :eyes: emoji to the Slack post so other maintainers know it is being assessed.
We do not want multiple maintainers to work on fulfilling the request.
1. Assess whether the merge request is urgent or not. If in doubt, ask the requestor for more details in the merge request about why it is urgent.

Next, ensure that all the following conditions are met:

1. The latest pipeline has completed less than 2 hours ago (although it is likely to have failed due to
gitlab-org/gitlab using
merged results pipelines).
1. All of the latest pipeline failures also happen on master.
1. There is a corresponding non-resolved broken master incidents.
See the "Triage DRI Responsibilities" steps above for more details.

Next, add a comment to the merge request mentioning that the merge request will be merged during a broken master, and link to the broken master incident. For example:

md

Merge request will be merged while master is broken.

Failure in <JOBURL> happens in master and is being worked on in <INCIDENTURL>.

Next, merge the merge request:

- If the "Merge" button is enabled (this is unlikely), then click it.
- Otherwise, you must:
 1. Unset the
"Pipelines must succeed" setting
for the gitlab-org/gitlab project.
 1. Click the "Merge" button.
 1. If the merge train is enabled, a warning will be displayed stating the code changes won't be validated by the merge train. Considering the criticality of the merge request it is acceptable to dismiss the warning.
 1. Set the "Pipelines must succeed" setting to be on again.

Broken master mirrors

master-broken-mirrors was created to remove duplicative notifications from the master-broken channel which provides a space for Release Managers and the Developer Experience teams to monitor failures for the following projects:

- <<https://gitlab.com/gitlab-org/security/gitlab>>

- <<https://dev.gitlab.org/gitlab/gitlab-ee>>

The master-broken-mirrors channel is to be used to identify unique failures for those projects and flaky failures are not expected to be retried/reacted to in the same way as master-broken.

Broken JiHu validation pipelines

We run JiHu validation pipelines in some of the merge requests, and it can be broken at times. When this happens, check [What to do when the validation pipeline failed](#) for more details.

Security Issues

Security issues are managed and prioritized by the security team. If you are assigned to work on a security issue in a milestone, you need to follow the Security Release process.

If you find a security issue in GitLab, create a confidential issue mentioning the relevant security and engineering managers, and post about it in security.

If you accidentally push security commits to gitlab-org/gitlab, we recommend that you:

1. Delete the relevant branch ASAP

1. Inform a release manager in releases. It may be possible to execute a garbage collection (via the Housekeeping task in the repository settings) to remove the commits.

For more information on how the entire process works for security releases, see the [documentation on security releases](#).

Regressions

A ~regression implies that a previously verified working functionality no longer works. Regressions are a subset of bugs. The ~regression label is used to imply that the defect caused the functionality to regress.

The label tells us that something worked before and it needs extra attention from Engineering and Product Managers to schedule/reschedule.

The regression label does not apply to bugs for new features for which functionality was never verified as working.

These, by definition, are not regressions.

A regression should always have the ~regression:xx.x label on it to designate when it was introduced. If it's unclear when it was introduced, the latest released version should be added.

Regressions should be considered high priority issues that should be solved as soon as possible, especially if they have severe impact on users. When identified in time, for example in a SaaS deployment, fixing them within the same milestone avoids their being included with that release.

Use of the ~regression label on MRs

For better efficiency, it's common for a regression to be fixed in an MR without an issue being created, either through reversion of the original MR or a code change. Regardless of whether there is an issue or not, the MR should also have the ~regression and ~regression:xx.x labels. This allows for trends to be accurately measured.

Basics

1. Start working on an issue you're assigned to. If you're not assigned to any issue, find the issue with the highest priority and relevant label you can work on, and assign it to yourself. You can use this query, which sorts by priority for the started milestones, and filter by the label for your team.

1. If you need to schedule something or prioritize it, apply the appropriate labels (see Scheduling issues).

1. If you are working on an issue that touches on areas outside of your expertise, be sure to mention someone in the other group(s) as soon as you start working on it. This allows others to give you early feedback, which should save you time in the long run.

1. If you are working on an issue that requires access to specific features, systems, or groups, open an access request to obtain access on staging and production for testing your changes after they are merged.

1. When you start working on an issue:

- Add the workflow::in dev label to the issue.
- Create a merge request (MR) by clicking on the Create merge request button in the issue.

This creates a MR with the labels, milestone and title of the issue. It also relates the just created MR to the issue.

- Assign the MR to yourself.
- Work on the MR until it is ready, it meets GitLab's definition of done, and the pipeline succeeds.

- Edit the description and click on the Remove the Draft: prefix from the title button.

- Assign it to the suggested reviewer(s) from Reviewer Roulette. If there are reviewers for multiple categories, for example: frontend, backend and database, assign all of them.

Alternatively, assign someone who specifically needs to review. When assigning, also @mention them in the comments, requesting a review.

- (Optionally) Unassign yourself from the MR. Some may find leaving the MR assigned to themselves easier to track the MRs they are responsible for by using the built in MR button/notification icon in the GitLab navigation bar.

- Change the workflow label of the issue to workflow::in review. If multiple people are working on the issue or multiple workflow labels might apply, consider breaking the issue up. Otherwise, default to the workflow label farthest away from completion.

- Potentially, a reviewer offers feedback and assigns back to the author.

- The author addresses the feedback and this goes back and forth until all reviewers approve the MR.

- After approving, the reviewer in each category unassigns themselves and assigns the suggested maintainer in their category.

- Maintainer reviews take place with any back and forth as necessary and attempts to resolve any open threads.

- The last maintainer to approve the MR, follows the Merging a merge request guidelines.

- (Optionally) Change the workflow label of the issue to workflow::verification, to indicate

all the development work for the issue has been done and it is waiting to be deployed and verified. We will use this label in cases where the work was requested to be verified by product OR we determined we need to perform this verification in production.

- When the change has been verified, change the workflow label to workflow::complete and close the issue.

1. You are responsible for the issues assigned to you. This means it has to ship with the milestone it's associated with. If you are not able to do this, you have to communicate it early to your manager and other stakeholders (e.g. the product manager, other engineers working on dependent issues). In teams, the team is responsible for this (see Working in Teams). If you are uncertain, err on the side of overcommunication. It's always better to communicate doubts than to wait.

1. You (and your team, if applicable) are responsible for:

- Ensuring that your changes apply cleanly to GitLab Enterprise Edition.
- The testing of a new feature or fix, especially right after it has been merged and packaged.
- Creating any relevant feature or API documentation
- Shipping secure code, (see Security is everyone's responsibility).

1. Once a release candidate has been deployed to the staging environment, please verify that your changes work as intended. We have seen issues where bugs did not appear in development but showed in production (e.g. due to CE-EE merge issues).

Be sure to read general guidelines about issues and merge requests.

Updating Workflow Labels Throughout Development

Team members use labels to track issues throughout development. This gives visibility to other developers, product managers, and designers, so that they can adjust their plans during a monthly iteration. An issue should follow these stages:

- workflow::in dev: A developer indicates they are developing an issue by applying the in dev label.
- workflow::in review: A developer indicates the issue is in code review and UX review by replacing the in dev label with the in review label.
- workflow::verification: A developer indicates that all the development work for the issue has been done and is waiting to be deployed, then verified.
- workflow::complete: A developer indicates the issue has been verified and everything is working by adding the workflow::complete label and closing the issue.

Workflow labels are described in our Development Documentation and Product Development Flow.

Working in Teams

For larger issues or issues that contain many different moving parts, you'll be likely working in a team. This team will typically consist of a backend engineer, a frontend engineer, a Product Designer and a product manager.

1. Teams have a shared responsibility to ship the issue in the planned release.

1. If the team suspects that they might not be able to ship something in time, the team should escalate / inform others as soon as possible. A good start is informing your manager.

1. It's generally preferable to ship a smaller iteration of an issue, than ship something a release later.

1. Consider starting a Slack channel for a new team, but remember to write all relevant information in the related issue(s). You don't want to have to read up on two threads, rather than only one, and Slack channels are not open to the greater GitLab community.

1. If an issue entails frontend and backend work, consider separating the frontend and backend code into separate MRs and merge them independently under feature flags. This will ensure frontend/backend engineers can work and deliver independently.

1. It's important to note that even though the code is merged behind a feature flag, it should still be production ready and continue to hold our definition of done.

1. A separate MR containing the integration, documentation (if applicable) and removal of the feature flags should be completed in parallel with the backend and frontend MRs, but should only be merged when both the frontend and backend MRs are on the master branch.

In the spirit of collaboration and efficiency, members of teams should feel free to discuss issues directly with one another while being respectful of others' time.

Convention over Configuration

Avoid adding configuration values in the application settings or in `gitlab.yml`. Only add configuration if it is absolutely necessary. If you find yourself adding parameters to tune specific features, stop and consider how this can be avoided. Are the values really necessary? Could constants be used that work across the board? Could values be determined automatically? See [Convention over Configuration](#) for more discussion.

Choosing Something to Work On

Start working on things with the highest priority in the current milestone. The priority of items are defined under labels in the repository, but you are able to sort by priority.

After sorting by priority, choose something that you're able to tackle and falls under your responsibility. That means that if you're a frontend developer, you work on something with the label frontend.

To filter very precisely, you could filter all issues for:

- Milestone: Started
- Assignee: None (issue is unassigned)
- Label: Your label of choice. For instance CI/CD, Discussion, Quality, frontend, or Platform
- Sort by priority

Use this [link](#) to quickly set the above parameters. You'll still need to filter by the label for your own team.

If you're in doubt about what to work on, ask your lead. They will be able to tell you.

Triaging and Reviewing Code from the rest of the Community

It's every developers' responsibilities to triage and review code contributed by the rest of the community, and work with them to get it ready for production.

Merge requests from the rest of the community should be labeled with the Community contribution label.

When evaluating a merge request from the community, please ensure that a relevant PM is aware of the pending MR by mentioning them.

This should be to be part of your daily routine. For instance, every morning you could triage new merge requests from the rest of the community that are not yet labeled Community contribution and either review them or ask a relevant person to review it.

Make sure to follow our Code Review Guidelines.

Working with GitLab.com

GitLab.com is a very large instance of GitLab Enterprise Edition. It runs release candidates for new releases, and sees a lot of issues because of the amount of traffic it gets. There are several internal tools available for developers at GitLab to get data about what's happening in the production system:

Performance Data

There is extensive monitoring publicly available for GitLab.com. For more on this and related tools, see the monitoring handbook.

Error Reporting

- Sentry is our error reporting tool
- log.gprd.gitlab.net has production logs
- prometheus.gitlab.com has alerts for the production team

Scheduling Issues

GitLab Inc has to be selective in working on particular issues. We have a limited capacity to work on new things. Therefore, we have to schedule issues carefully.

Product Managers are responsible for scheduling all issues in their respective product areas, including features, bugs, and tech debt. Product managers alone determine the prioritization, but others are encouraged to influence the PMs decisions. The UX Lead and Engineering Leads are responsible for allocating people making sure things are done on time. Product Managers are not responsible for these activities, they are not project managers.

Direction issues are the big, prioritized new features for each release. They are limited to a small number per release so that we have plenty of capacity to work on other important issues, bug fixes, etc.

If you want to schedule an issue with the Seeking community contributions label, please remove the label first.

Any scheduled issue should have a team label assigned, and at least one type label.

Requesting Something to be Scheduled

To request scheduling an issue, ask the responsible product manager

We have many more requests for great features than we have capacity to work on. There is a good chance we'll not be able to work on something. Make sure the appropriate labels (such as customer) are applied so every issue is given the priority it deserves.

Product Development Timeline

Teams (Product, UX, Development, Quality) continually work on issues according to their respective workflows.

There is no specified process whereby a particular person should be working on a set of issues in a given time period.

However, there are specific deadlines that should inform team workflows and prioritization.

With the monthly release date being the third Thursday of the release month, the code cut-off is the Friday prior.

The next milestone begins the Saturday after code cut-off.

All other important dates for a milestone are relative to the release date:

- Monday, 19 days before the milestone begins:
 - Draft of the issues that will be included in the next release (released next month).
 - Start capacity and technical discussions with engineering/UX.
 - Assess error budgets to determine feature/reliability balance.
 - Development engineering manager provides product manager with prioritization input for ~type::maintenance issues per cross-functional prioritization
 - Quality provides product manager with prioritization input for ~type::bug issues per cross-functional prioritization
- Monday, 12 days before the milestone begins:
 - Product manager, taking into consideration prioritization input from development EM, Quality, and UX to create a plan of issues for the upcoming milestone.
 - Release scope is finalized. In-scope issues marked with milestone %x.y; label ~deliverable applied.
 - Kickoff document is updated with relevant items to be included.
- Monday, 5 days before the milestone begins:
 - Release scope is finalized. In-scope issues marked with milestone %x.y; label ~deliverable applied.
 - Kickoff document is updated with relevant items to be included.
- Monday immediately after the milestone begins: Kick off! ·

- Company Kickoff call live streamed.
- Development on milestone begins.
- Monday, 9 days after the milestone begins:
 - The development lead for each stage/section coordinates a stage/section level review with the quad cross-functional dashboard review process. After the stages/section level reviews are complete, the VP of Development coordinates a summary review with the CTO, VP of Product, VP of UX, and VP of Quality.
- Wednesday, 11 days after the milestone begins:
 - GitLab Bot opens Group Retrospective issue for the current milestone.
- Friday, the day the milestone ends:
 - Issues for milestone completed, with docs, and have been merged into master.
 - Feature flags should be flipped from default off to default on after verification to be in the release. See feature flags.
 - Merging by the milestone code cut-off (Friday) does not guarantee that the feature will be in the release. See release ti